MODULE 15 · DOMAIN 2 - RELIABILITY AND BUSINESS CONTINUITY

Scaling the Data Tier

Read replicas, Aurora serverless, DynamoDB capacity, ElastiCache - and why on-demand still throttles.

DOMAIN 2 of 5 EXAM WEIGHT 22% MODULE 15 of 35

admin@sthithapragna.com

Three axes: make it **bigger**, add **more of them**, or make the work **smaller**. Most scaling questions are asking which axis, not which service.

Vertical and horizontal, for databases

- **Vertical**: a bigger instance class. Simple, effective, and it requires a restart or a failover, plus it has a maximum size.
- **Horizontal reads**: read replicas. Straightforward, and it introduces replication lag your application must tolerate.
- **Horizontal writes** are the hard one. A relational database has one writer, so writes scale vertically or by sharding the application.
- On Multi-AZ, resizing modifies the **standby first**, then fails over, so the disruption is one failover rather than an outage.

> ON THE EXAM
>
> "Scale writes" in a relational database is usually a design question, not a configuration one. Reads are easy; writes are not.

Read replicas in depth

- Asynchronous replication from the source. Up to **15 for Aurora**, and typically **5 per source** for RDS engines.
- Can live in the **same AZ, another AZ, or another Region**. Cross-Region replicas also serve as a disaster recovery position.
- **Promotion is manual** and breaks replication permanently. The promoted replica becomes an independent database.
- A replica can itself be **Multi-AZ**, and RDS replicas can be **cascaded**, which adds lag at every hop.

> ON THE EXAM
>
> Applications must be changed to use a replica. Read traffic does not move there by itself, which is the difference from an Aurora reader endpoint.

Replica lag, and what causes it

- **ReplicaLag** is the metric. Alarm on it, because an application reading stale data usually fails quietly rather than loudly.
- Causes: heavy writes on the source, a large transaction, an undersized replica, or a long-running query blocking apply.
- **Aurora lag is typically well under a second**, because replicas read the shared storage volume rather than replaying a log.
- Lag also **bounds failover time**, as module fourteen noted. A lagging replica is slow to promote.

> ON THE EXAM
>
> "Users see their own change disappear after saving" is replica lag with read-after-write on a replica. Route those reads to the writer.

Aurora replica auto scaling

- **Application Auto Scaling** adds and removes Aurora replicas based on a target: average CPU, or average connections.
- You set a **minimum and maximum replica count**, and a target value. Same target tracking model as module twelve.
- New replicas join the **reader endpoint** automatically, so read traffic spreads with no application change.
- Scaled replicas are added with a **promotion tier of 15**, so they are the last candidates for promotion on failover.

> ON THE EXAM
>
> This is EC2 Auto Scaling's model applied to database readers. Note it is Application Auto Scaling, which module twelve distinguished carefully.

Naming: v2 is now Aurora serverless

- **In April 2026 AWS renamed Aurora Serverless v2 to simply Aurora serverless.** No action required, same product.
- The **exam guide's in-scope list still says Aurora Serverless v2**. Learn the capability, and recognise both names.
- **Aurora Serverless v1 was a different architecture** and is deprecated. Do not confuse it with either name above.
- v1 doubled capacity in steps and paused disruptively. The current product scales in fine increments without dropping connections.

> ON THE EXAM
>
> Three names, two products. v1 is gone, and v2 is now called Aurora serverless. Exam material will use the older name for a while yet.

How Aurora serverless scales

- Capacity is measured in **Aurora Capacity Units**, each roughly **2 GiB of memory** with matching CPU and network.
- Adjusts in increments as small as **0.5 ACU**, up to a maximum of **256 ACUs**, without dropping connections.
- Recent platform versions **add 12 ACUs within a second** and keep climbing, which suits sharp unpredictable bursts.
- It can scale to **zero ACUs** and pause, then resume on the next connection. That resume is a cold start.

> ON THE EXAM
>
> Set a minimum above zero for production. Scale-to-zero is a development and test feature, and the resume latency is real.

The Aurora serverless limitation people miss

- **Aurora Auto Scaling, which adds reader replicas by count, is not supported on Aurora serverless.**
- Serverless scales each instance **vertically**, by growing its ACUs. It does not add readers for you.
- To scale reads horizontally on serverless, you **add reader instances yourself**, each of which then scales its own capacity.
- **Mixed clusters are allowed**: a provisioned writer with serverless readers, or the reverse.

> ON THE EXAM
>
> "Serverless means it handles everything automatically" is the wrong mental model. It automates instance size, not instance count.

Provisioned Aurora or Aurora serverless?

PROVISIONED WHEN

Load is steady

- Predictable, sustained throughput
- Cheaper per unit of work at steady high load
- Reserved instances available for further savings
- Aurora Auto Scaling can add readers by CPU

SERVERLESS WHEN

Load is spiky or unknown

- Bursty, intermittent or unpredictable traffic
- Development and test, where scale to zero pays
- No capacity planning, fine-grained billing
- Not automatically cheaper at sustained high load

THE TIEBREAKER

Do the arithmetic. A database busy 24 hours a day is usually cheaper provisioned, and serverless wins on the workload that is idle most of the week.

DynamoDB capacity, revisited for growth

- **On-demand is now the default and AWS's recommended mode.** It scales automatically and charges per request.
- **Provisioned with auto scaling** is cheaper for steady, predictable load, and reserved capacity cheaper still.
- **Maximum throughput for on-demand tables** caps reads or writes per second, protecting the budget and downstream services.
- Default account limits are **40,000 read and 40,000 write units per table**, raisable on request.

> ON THE EXAM
>
> The maximum throughput setting is the answer to "a runaway script cost us thousands overnight" and to "the database overwhelmed our downstream API".

Warm throughput: why on-demand still throttles

- Every table, index and replica has a **warm throughput**: what it can serve **instantly**, without waiting to scale.
- A new on-demand table starts at about **12,000 reads and 4,000 writes per second**.
- Beyond that, DynamoDB **does** scale, but scaling takes time, and requests arriving faster than it scales are **throttled**.
- Warm throughput **only ratchets upward**. **Pre-warming** raises it ahead of a launch, a sale or a batch migration.

> ON THE EXAM
>
> "On-demand means it never throttles" is wrong. It means it always eventually scales. Warm throughput is the gap between those two statements.

DynamoDB auto scaling, and why it looks slow

- Provisioned auto scaling uses **Application Auto Scaling** with CloudWatch alarms, targeting a utilisation percentage.
- It scales **up** after utilisation exceeds the target for about **2 consecutive minutes**.
- It scales **down** only after utilisation is **20 percent or more below target for 15 consecutive minutes**.
- So a sharp spike is throttled for a couple of minutes before capacity arrives. **Burst capacity** absorbs some of that, briefly.

> ON THE EXAM
>
> Provisioned auto scaling is designed for gradual change. For genuinely sudden spikes, on-demand or pre-warming is the answer, not a lower target.

Indexes have their own capacity

- A **global secondary index** has its own throughput. In provisioned mode you scale it separately from the table.
- **A throttled GSI throttles writes to the base table**, because a write must succeed on both.
- Projecting fewer attributes into a GSI reduces its write cost, since every base-table write updates every projecting index.
- **Local secondary indexes** share the table's capacity and must be created with the table. They cannot be added later.

> ON THE EXAM
>
> "The table has plenty of capacity but writes are throttled" is nearly always a global secondary index that does not.

ElastiCache: node-based or Serverless?

NODE-BASED WHEN

You want control

- Choose node types and counts yourself
- Shards for write scaling, replicas for read scaling
- Reserved node discounts available
- Predictable hourly cost regardless of traffic

SERVERLESS WHEN

You want it to just work

- Scales automatically, no capacity planning
- Billed per ECPU plus stored GB-hours
- Very low floor: about 100 MB minimum for Valkey
- Can exceed node-based cost at very high request rates

THE TIEBREAKER

AWS recommends Valkey for new deployments. It is roughly 20 percent cheaper node-based and 33 percent cheaper serverless than Redis OSS.

Caching strategies

- **Lazy loading**: read the cache, and on a miss read the database and populate. Only requested data is cached.
- Its costs: a **three-trip penalty on every miss**, and stale data until the entry expires.
- **Write-through**: update the cache whenever you write. Data is never stale, and the cache fills with data nobody reads.
- **A TTL on every key** is the practical safety net. It bounds staleness whichever strategy you use.

> ON THE EXAM
>
> Most production systems use lazy loading plus a TTL, and add write-through only for the small set of keys where staleness is unacceptable.

Scale the database, or add a cache?

SCALE THE DATABASE WHEN

The work is genuinely new

- Writes are the bottleneck
- Queries are diverse, with little repetition
- Every read must be strongly consistent
- Data changes faster than it is read

ADD A CACHE WHEN

The same work repeats

- A small set of items dominates reads
- Reads vastly outnumber writes
- Some staleness is acceptable
- Expensive queries with stable results

THE TIEBREAKER

A cache is the cheapest scaling there is when the read pattern is skewed. It is useless when every query is different or every read must be current.

An Aurora PostgreSQL cluster is configured as Aurora serverless with a minimum of 2 and a maximum of 64 ACUs. Read traffic has grown steadily and the single instance now sits near its maximum during business hours. Reads are 90 percent of the workload.

The team expects Aurora Auto Scaling to add reader instances and is confused that it has not.

What should the team do?

A Enable Aurora Auto Scaling on the cluster to add readers automatically

B Add reader instances to the cluster and send read traffic to the reader endpoint

C Raise the maximum to 256 ACUs and leave the topology unchanged

D Convert the cluster to provisioned Aurora with a larger instance class

◆ PAUSE AND DECIDE

What should the team do?

A Enable Aurora Auto Scaling

Not supported on Aurora serverless. It adds readers by count; serverless scales each instance's capacity instead.

B Add readers, use the reader endpoint

Reads are 90 percent of the load, so horizontal read scaling is the right axis. Each added reader then scales its own ACUs.

C Raise the maximum to 256

Buys headroom on one instance and keeps every read on the writer. It is vertical scaling applied to a horizontal problem.

D Convert to provisioned

A large change that would let Aurora Auto Scaling work, at the cost of losing fine-grained scaling. Adding readers is far less disruptive.

THE SIGNAL

Serverless automates instance size, not instance count. Ninety percent reads is the clue that count is what needs to change.

A retailer moves a DynamoDB table to on-demand capacity mode before a flash sale, reasoning that on-demand scales automatically and cannot throttle. The table normally serves about 800 reads per second.

At the start of the sale, traffic jumps to roughly 60,000 reads per second within thirty seconds. Customers see errors, and CloudWatch shows ReadThrottleEvents for several minutes.

What explains this, and what should they have done?

A On-demand tables have a fixed 40,000 read limit that cannot be raised

B The table's warm throughput was too low for the spike. Pre-warm it before the event, or request a higher throughput ahead of time

C Switch back to provisioned mode with auto scaling and a high target utilisation

D Add a global secondary index to spread the read load

◆ PAUSE AND DECIDE

What explains this, and what should they have done?

A Fixed 40,000 limit

There is a default account-level limit around that figure, and it is raisable on request, so "cannot be raised" makes this wrong.

B Warm throughput too low; pre-warm

On-demand scales, but not instantly. Warm throughput is what it serves immediately, and pre-warming raises it ahead of a known event.

C Provisioned with auto scaling

Worse for this shape. Provisioned auto scaling takes about two minutes to react, which is slower than on-demand, not faster.

D Add a GSI

An index serves different query patterns. It does not increase capacity for the same reads, and it adds write cost.

THE SIGNAL

The retailer's stated reasoning is the distractor. On-demand means it always eventually scales, not that it never throttles.

> A provisioned DynamoDB table has 20,000 write capacity units and consumes about 9,000. Writes are being throttled intermittently. The table has three global secondary indexes; one of them is provisioned at 2,000 write units and is consistently at its limit.
>
> The partition key is a high-cardinality UUID and traffic is evenly distributed.

What is the cause?

A. A hot partition on the base table

B. The throttled GSI is throttling base-table writes, because a write must succeed on the table and every projecting index

C. Auto scaling is scaling down too aggressively

D. The table needs to be switched to on-demand mode

◆ PAUSE AND DECIDE

What is the cause?

A **Hot partition**

Explicitly ruled out. A high-cardinality key with evenly distributed traffic is the opposite of a hot partition.

B **The GSI is the bottleneck**

A write must land on the base table and every index that projects the written attributes. A throttled index throttles the write.

C **Auto scaling too aggressive**

The base table is at 45 percent of its provisioned capacity, so nothing about the base table's scaling is the constraint.

D **Switch to on-demand**

Would mask it by removing provisioning, at higher cost, without addressing an index that is genuinely undersized.

THE SIGNAL

Compare this with module ten's hot-partition scenario. Same symptom, opposite cause, and the partition key detail is what separates them.

A product catalogue API serves 40,000 reads per second against RDS for MySQL. About 200 popular products account for 85 percent of requests, and the catalogue is updated a few times a day. The database CPU is saturated and the team has already scaled the instance twice.

A brief delay in reflecting catalogue updates is acceptable.

What is the most cost-effective way to scale this?

A Add ElastiCache in front of the database, using lazy loading with a TTL

B Add five read replicas and distribute reads across them

C Scale the instance class up again

D Migrate the catalogue to DynamoDB with on-demand capacity

◆ PAUSE AND DECIDE

What is the most cost-effective way to scale this?

A ElastiCache, lazy loading with a TTL

Highly skewed reads, rare writes, and staleness tolerated. A cache absorbs 85 percent of the load for a fraction of the cost of database capacity.

B Five read replicas

Works, and it is five more database instances to pay for and operate, all repeatedly answering the same 200 questions.

C Scale up again

The team has already done this twice, which is the scenario telling you the vertical axis is exhausted.

D Migrate to DynamoDB

A large migration and an application rewrite to solve a read-skew problem that a cache solves in an afternoon.

THE SIGNAL

Three properties in the setup are the textbook caching signature: skewed reads, infrequent writes, and tolerated staleness.

A reporting dashboard reads from an RDS for PostgreSQL read replica. Users report that after saving a change they sometimes do not see it, and refreshing a few seconds later shows it correctly. The application writes to the primary and reads from the replica for all queries.

ReplicaLag averages 1.5 seconds and peaks at 8 seconds during batch loads.

What should the team change?

A Route reads that immediately follow a write to the primary, and leave other reads on the replica

B Increase the replica's instance class to eliminate lag entirely

C Convert the replica into a Multi-AZ standby

D Add a second read replica and load-balance between them

◆ PAUSE AND DECIDE

What should the team change?

A Read-after-write goes to the primary

Asynchronous replication always has some lag. Routing only the reads that must be current to the writer keeps the offload and fixes the symptom.

B Bigger replica instance

Reduces lag; cannot remove it. Asynchronous means the replica is always at least a little behind, so the bug would become rarer and stay.

C Convert to a Multi-AZ standby

The standby serves no reads at all, so this removes the read offload entirely and solves nothing about the dashboard.

D A second replica

Doubles read capacity and doubles the number of places serving stale data. It makes the symptom no less likely.

THE SIGNAL

This is a read-after-write consistency problem, not a capacity problem. No amount of extra replica capacity makes asynchronous replication synchronous.

The five things to carry forward

- **Bigger, more of them, or less work.** Decide the axis before the service.
- **Aurora Auto Scaling adds readers and is not supported on Aurora serverless**, which scales size rather than count.
- **On-demand DynamoDB still throttles** beyond its warm throughput. Pre-warm before a known spike.
- **A throttled GSI throttles base-table writes.** Check indexes when the table looks comfortable.
- **Skewed reads, rare writes, tolerable staleness: cache.** Read-after-write goes to the writer.

> ON THE EXAM
>
> Next: backup and restore - AWS Backup, snapshots, point-in-time recovery, versioning and Vault Lock.